



Definição do Trabalho 1: Transferência de Arquivos Distribuída

Este trabalho tem como objetivo exercitar a comunicação por troca de mensagens via Sockets entre processos distribuídos, explorando protocolos fundamentais e serviços *web*.

Como tarefa, você deverá implementar um serviço de *backup* de arquivos, capaz de realizar o *upload* de arquivos de forma confiável. O projeto consiste em três programas independentes:

- *cliente (sender)*: executável por linha de comando, espera como um argumento um endereço IP do *servidor* e um nome de arquivo (e.g. `/path/to/file.txt`) ao qual se deseja transferir.
- *servidor de upload (receiver)*: espera como argumentos opcionais uma porta e caminho de diretório onde armazenará arquivos recebidos por clientes concorrentes. Pode-se utilizar valores padrão em caso não informado (e.g. `“:9000”` e `“$HOME”`).
- *servidor web*: espera como argumento de entrada um caminho de diretório onde realiza a leitura de arquivos, disponibilizando uma API REST com um único método `GET /files`, que retorna uma listagem de arquivos disponíveis em formato JSON com seus respectivos *status* de transferência (disponível/em transferência).

O programa deve ser capaz de transferir arquivos de até 1GB de forma confiável, ou seja, sem perder pacotes, e construído sobre o **protocolo UDP** utilizando o algoritmo de *Stop and Wait*. Em caso de interrupção do processo cliente, o programa também deve ser capaz de retomar a transferência do estado de onde parou, sem resultar em arquivos duplicados ou corrompidos no servidor. Além disso, como limitação de recurso, deve-se carregar em memória tamanhos de arquivos de até 32 kB (32,768 bytes) por vez.

Pode-se assumir o envio de arquivos somente por rede local (LAN), sem a necessidade de lidar com segurança (criptação).

Requisitos específicos e avaliação

- Implementar os processos *cliente* e *servidor de upload* utilizando a **linguagem C** e Sockets Berkeley, sem o uso de outras bibliotecas adicionais para transferência de arquivos¹. O *servidor web* pode ser implementado em linguagem de programação de livre escolha;
- Deve ser entregue o código com um documento de descrição (arquivo README) e detalhes para compilação e execução de cada processo;
- Entregar relatório redigido em **um arquivo .pdf à parte** respondendo as perguntas:
 1. Como sua implementação lida com a retransmissão e garantia de entrega dos pacotes? Exemplifique com um caso de uso e diagrama, ilustrando o cenário de falha do processo cliente.
 2. Como sua implementação lida com a transmissão de arquivos por clientes concorrentes? Exemplifique com um caso de uso e diagrama.
 3. Caso fosse permitida a implementação da transferência de arquivos sobre o protocolo TCP, em quais aspectos você acredita que simplificaria a solução?
- Defender implementação em sala de aula.

¹ Também não é permitido a invocação de comandos nativos (e.g. `scp`, `fsync`) via primitivas de `system()`, `popen()`, `exec()` ou similares.

Entrega

O trabalho pode ser realizado em grupos de até 3 participantes. As defesas de código ocorrerão em laboratório, em horário de aula, seguindo o cronograma da disciplina, com o agendamento de grupos via Moodle.

O código-fonte e relatório .pdf devem ser enviados pelo Moodle para análise e avaliação.

O nome dos membros do grupo deve aparecer nos artefatos gerados (código e relatório). Nomes que não estejam explícitos nos artefatos entregues não serão considerados na avaliação. Certifique-se de nomear o arquivo .zip adequadamente seguindo o formato: **Grupo1-NomeA-NomeB.zip**

Dicas

- Podem ser enviadas outras informações (metadados) além dos próprios arquivos.
- Ler sobre protocolo *Stop and Wait*
 - <https://www.geeksforgeeks.org/computer-networks/stop-and-wait-arq/>
 - https://en.wikipedia.org/wiki/Stop-and-wait_ARQ